

Algorithms and data structures I.

Queue

Queue with Arrays

- ◊ FIFO (First In First Out)
- ◊ Overall time complexity: $O(1)$
- ◊ Queue in an array:
 - ◊ Cyclic
 - ◊ First element in the queue: $Z[k]$ (if not empty)
 - ◊ Number of elements: n
 - ◊ First free slot $Z[k+n \bmod m]$

Queue
$-Z : \mathcal{T}[]$ // $\mathcal{T}$ is some known type
$-n : \mathbb{N}$ // $n \in 0..Z.length$ is the actual length of the queue
$-k : \mathbb{N}$ // $k \in 0..(Z.length - 1)$ is the starting position of the queue in array Z
$+ \text{Queue}(m : \mathbb{N}) \{ Z := \text{new } \mathcal{T}[m] ; n := 0 ; k := 0 \}$ // create an empty queue
$+ \text{add}(x : \mathcal{T})$ // join x to the end of the queue
$+ \text{rem}() : \mathcal{T}$ // remove and return the first element of the queue
$+ \text{first}() : \mathcal{T}$ // return the first element of the queue
$+ \text{length}() : \mathbb{N} \{ \text{return } n \}$
$+ \text{isFull}() : \mathbb{B} \{ \text{return } n = Z.length \}$
$+ \text{isEmpty}() : \mathbb{B} \{ \text{return } n = 0 \}$
$+ \sim \text{Queue}() \{ \text{delete } Z \}$
$+ \text{setEmpty}() \{ n := 0 \}$ // reinitialize the queue

Queue with Arrays

Queue::add($x : \mathcal{T}$)

$n < Z.length$

$Z[(k + n) \bmod Z.length] := x$

$n++$

fullQueueError

Queue::rem() : $\mathcal{T}$

$n > 0$

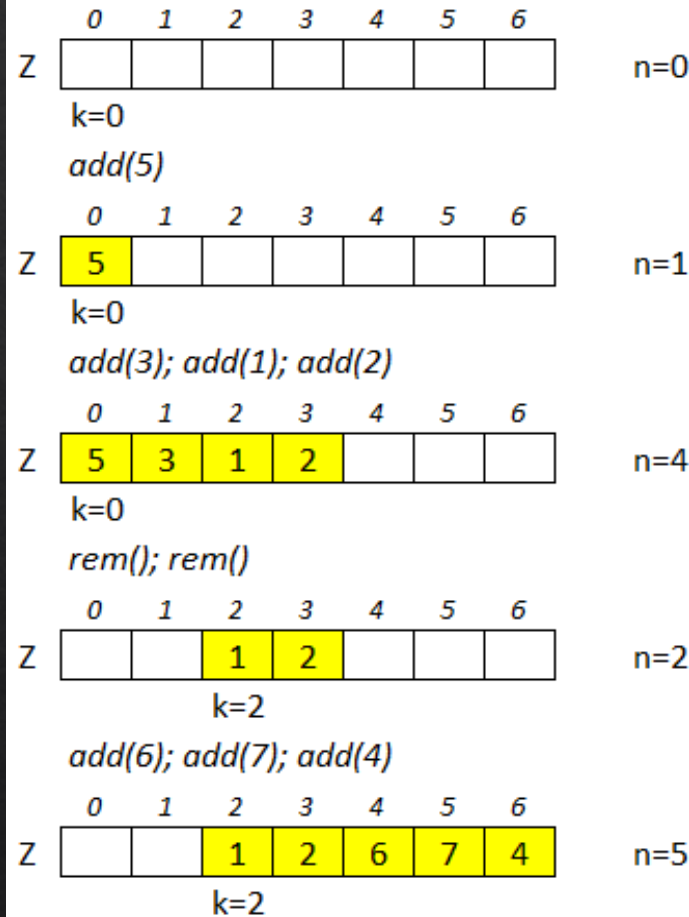
$n--$

$i := k$

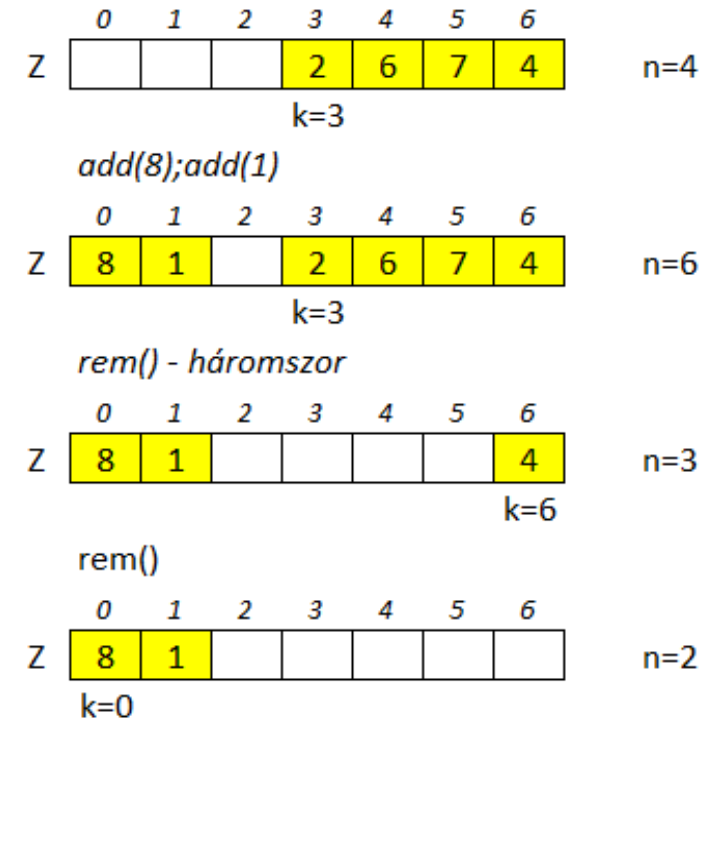
$k := (k + 1) \bmod Z.length$

return $Z[i]$

emptyQueueError



rem()



Queue - Linked Representation

◆ Queue methods:

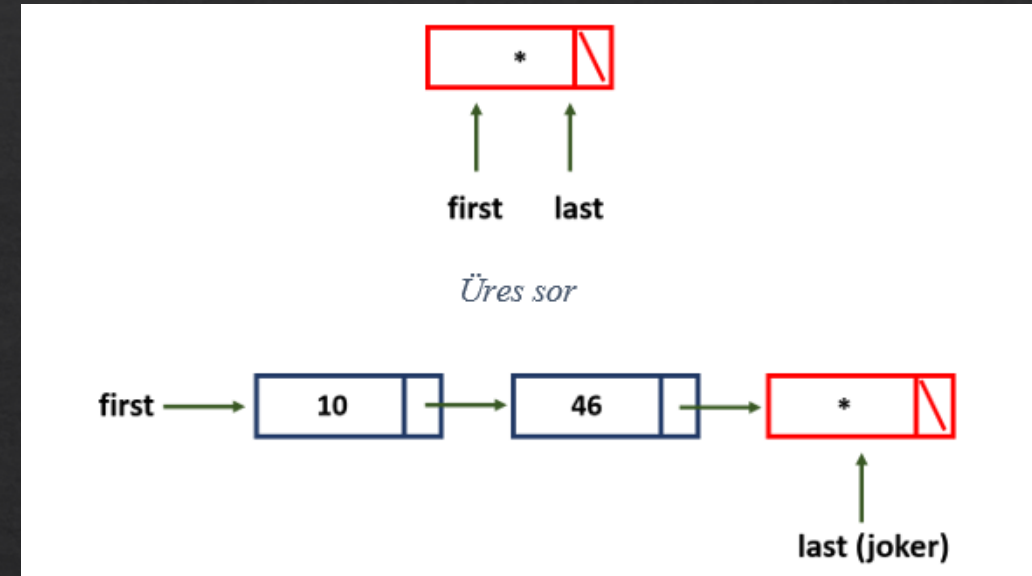
- `rem() : T`
- `add(x : T)`
- `first() : T`
- `length() : N`
- `isEmpty() : B`
- `setEmpty()`

Overall time complexity for arrays: $O(1)$
Except if we allow the queue to extend when it becomes full.

It's not an issue to a linked representation!

Queue – Simple Linked Representation

- ♦ Idea: Linked list with two pointers to first and last.
- ♦ It contains a 'joker' element at the end as a “next node to be inserted here” element.
- ♦ Whenever we add a new element we link it before this joker element.
- ♦ The 'last' pointer points to the joker.
- ♦ The 'joker' element is like a reversed header, this way the list is never empty.

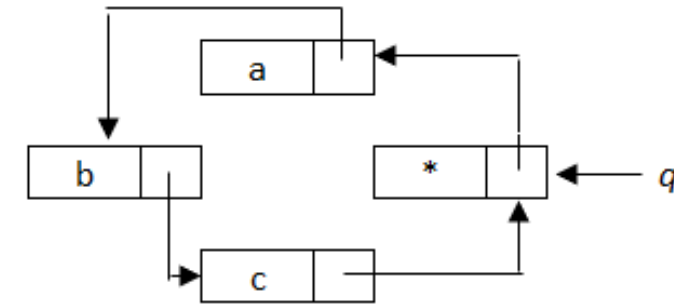
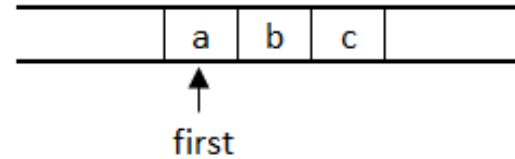


Queue	
-first, last: E1*	// New pointers pointing to the first and last elements
-size: N	
+ Queue() + add(x: T) // Adding new element at the end + rem(): T // Removing the first element + first() : T // Query the first element + length(): N + isEmpty(): B + ~Queue() + setEmpty()	

Queue – Cyclic Linked Representation

- ◇ This works similarly, having a joker (q) element, but this construction is cyclic
- ◇ Only a single pointer is needed, which points to the joker element. If the list is cyclic, the joker points to the first element, and therefore we can reach it instantly as well as insert an element in place of the joker instantly.

Q:



Queue

-q: $E1^*$ // A single pointer pointing at the joker
-size: N

+ Queue()
+ add(x: T) // Adding new element at the end
+ rem(): T // Removing the first element
+ first(): T // Query the first element
+ length(): N
+ isEmpty(): B
+ ~Queue()
+ setEmpty()

Queue – Cyclic List Stuctograms

Queue::Queue()

```
q := new E1
q->next := q
size := 0
```

Queue::rem(): T

size = 0	
ERROR	r := q->next->next
	x := q->next->key
	delete q->next
	q->next := r
	size := size-1
	return x

Queue::add(x: T)

```
r := q->next
q->key := x
q->next := new E1
q := q->next
q->next := r
size := size+1
```

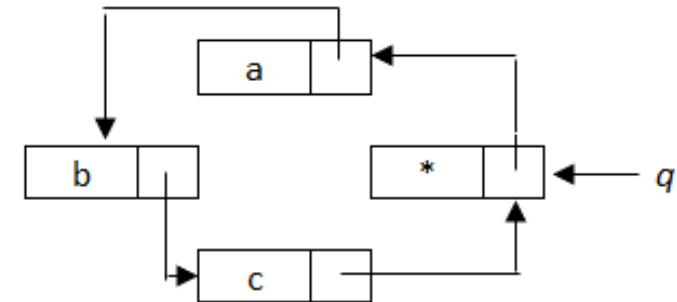
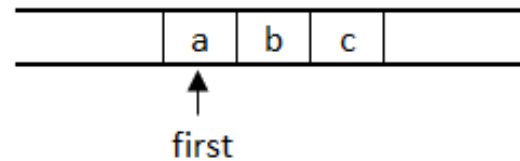
Queue::setEmpty()

```
p := q->next
  p ≠ q
    r := p
    p := p->next
    delete r
  q->next := q
  size := 0
```

Queue::~~Queue()

```
p := q->next
  p ≠ q
    r := p
    p := p->next
    delete r
  delete q
```

Q:

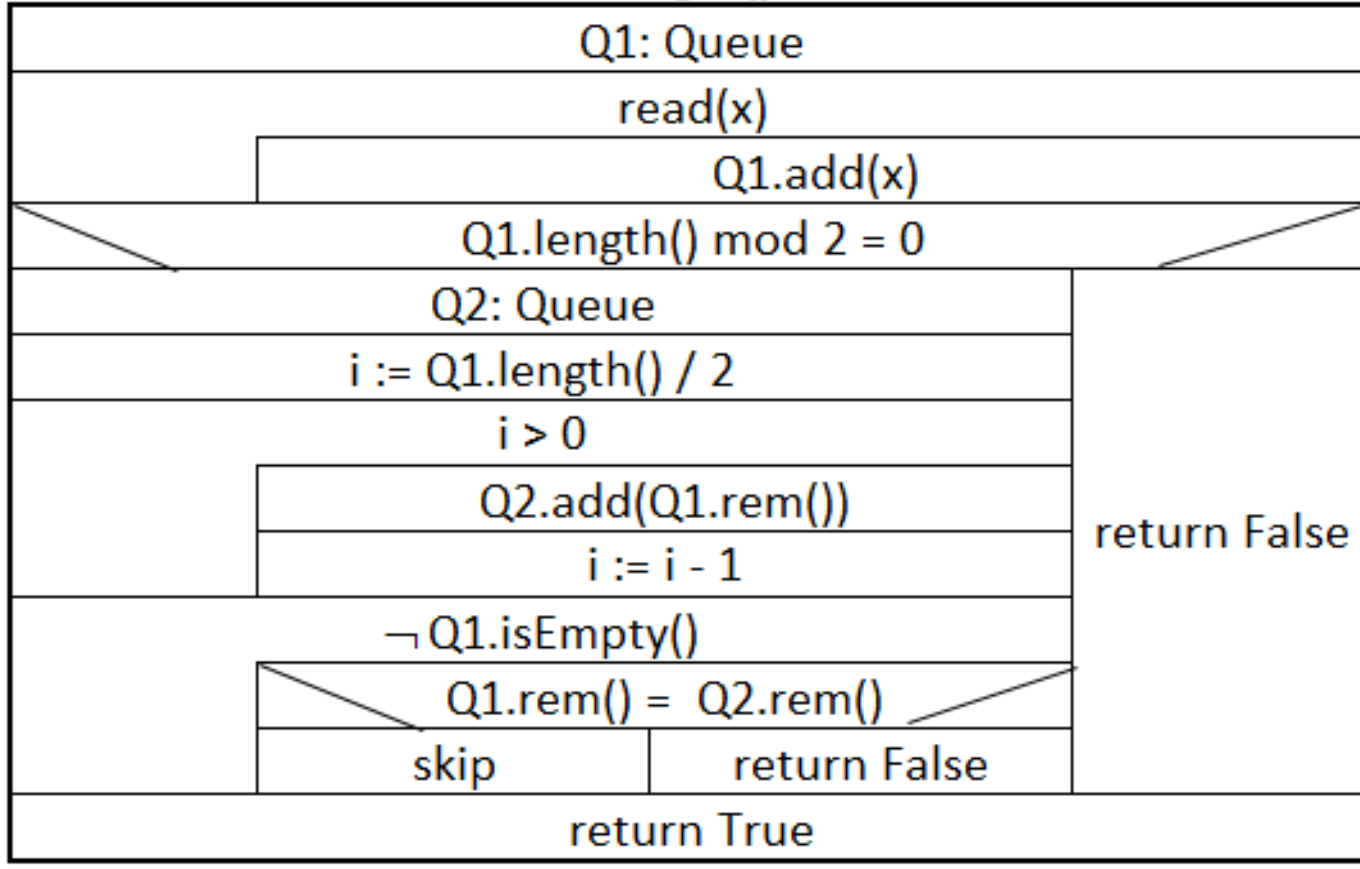


Problem: Doubled words

- ◇ Decide if a given input text is 'stuttery' or not.
- ◇ For example
 - ◇ ""
 - ◇ „aa”
 - ◇ „appleapple”
- ◇ Idea:
 - ◇ Read the whole text into a queue.
 - ◇ If the length is odd, it's not stuttery.
 - ◇ If even, copy one half into another queue and compare.

Solution: Doubled word

Doubled words (): B



- ◇ Init queues
- ◇ Read the text into Q1
- ◇ Check if odd
- ◇ Put one half of Q1 into Q2
- ◇ Compare the queues: If equals, then all good, continue.
- ◇ Return if the queue is empty, that is the answer to the problem.

Problem: Pascal with Queue

1			1					1.
		1		1				2.
	1		2		1			3.
1		3		3		1		4.

Pascal(k: N ; Q: Queue)	
	Q.setEmpty()
	Q.add(1)
	i := 2 to k
	e := 0
	j := 1 to i-1
	Q.add(e + Q.first())
	e := Q.rem()
	Q.add(1)